LAPORAN PRAKTIKUM

STRUKTUR DATA

MODUL 4

SINGLY LINKED LIST



Disusun oleh:

Salsadilla Hanny Azizah

109082500014

S1IF-13-02

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

TAHUN 2026

LATIHAN SOAL MODUL

Link Repo Github Modul 4 : salsadha/PRAKTIKUMSDT

1. **Soal 1**

   Source Code

```
#include <iostream>
#include <string>

#define Nil NULL

using namespace std;

// Tipe data bentukan untuk informasi nasabah
struct Nasabah {
   string nama;
};

typedef Nasabah infotype;
typedef struct ElmList *address;

struct ElmList {
   infotype info;
   address next;
};

struct List {
   address First;
};

// Prototipe primitif
void CreateList(List &L);
address alokasi(infotype x);
void dealokasi(address &P);
void insertLast(List &L, address P);
void deleteFirst(List &L, address &P);
void printInfo(List L);

int main() {
   List L;
   CreateList(L);
   int pilihan;
```

```
    do {
        cout << "\n=== MENU ANTREAN NASABAH BANK ===" << endl;
        cout << "1. Tambah Nasabah (Insert Last)" << endl;
        cout << "2. Layani Nasabah (Delete First)" << endl;
        cout << "3. Keluar" << endl;
        cout << "Pilihan: ";
        cin >> pilihan;

        if (pilihan == 1) {
            Nasabah n;
            cout << "Masukkan Nama Nasabah: ";
            cin >> n.nama;
            address P = alokasi(n);
            insertLast(L, P);
            printInfo(L);
        } else if (pilihan == 2) {
            if (L.First == Nil) {
                cout << "Antrean kosong, tidak ada nasabah untuk dilayani." << endl;
            } else {
                address P;
                deleteFirst(L, P);
                cout << "Melayani: " << P->info.nama << endl;
                dealokasi(P);
                printInfo(L);
            }
        }
    } while (pilihan != 3);

    return 0;
}

void CreateList(List &L) {
    L.First = Nil;
}

address alokasi(infotype x) {
    address P = new ElmList;
    P->info = x;
    P->next = Nil;
    return P;
}

void dealokasi(address &P) {
    delete P;
    P = Nil;
```

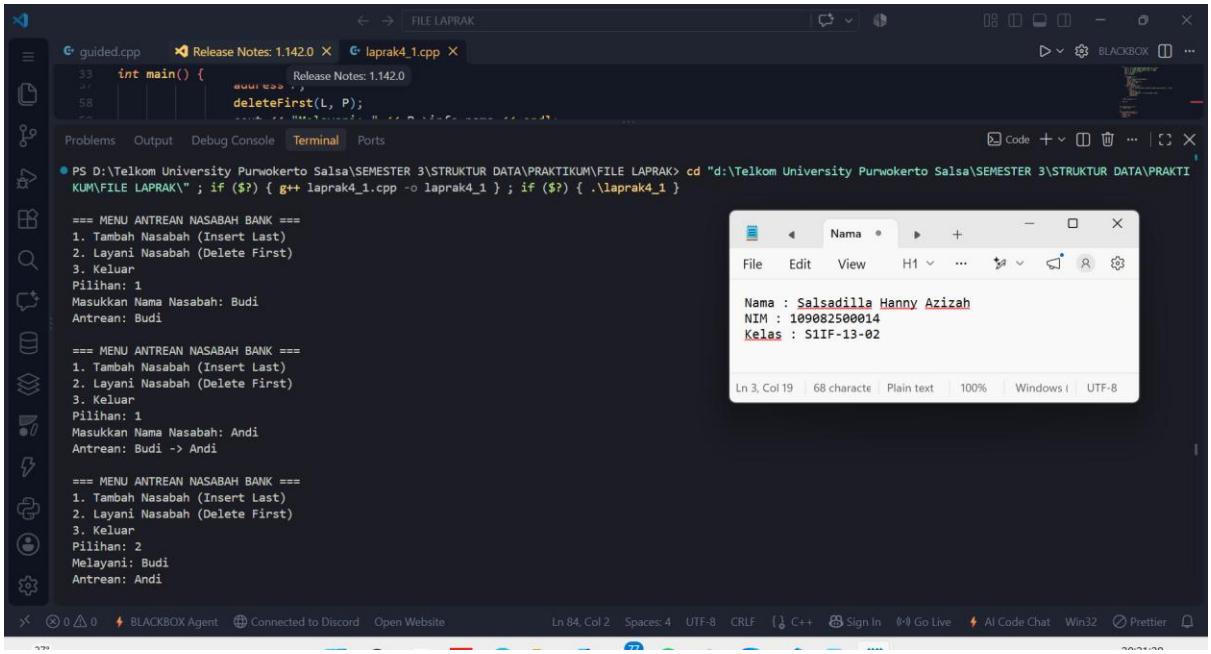
```
}

void insertLast(List &L, address P) {
   if (L.First == Nil) {
      L.First = P;
   } else {
      address Q = L.First;
      while (Q->next != Nil) {
         Q = Q->next;
      }
      Q->next = P;
   }
}

void deleteFirst(List &L, address &P) {
   P = L.First;
   L.First = L.First->next;
   P->next = Nil;
}

void printInfo(List L) {
   address P = L.First;
   cout << "Antrean: ";
   if (P == Nil) {
      cout << "(Kosong)" << endl;
      return;
   }
   while (P != Nil) {
      cout << P->info.nama;
      if (P->next != Nil) {
         cout << " -> ";
      }
      P = P->next;
   }
   cout << endl;
}
```

Screenshoot program



Deskripsi program

Program ini dibuat untuk mengimplementasikan struktur data *Singly Linked List* dalam mensimulasikan sistem antrean nasabah di sebuah bank. Alur kerja program menggunakan metode insertLast untuk menambahkan data nasabah baru di posisi paling belakang antrean, serta menggunakan metode deleteFirst untuk menghapus atau melayani nasabah yang berada di urutan paling depan. Program berjalan secara interaktif menggunakan perulangan do-while dan percabangan if-else hingga pengguna memilih menu keluar.

Penjelasan kode program:

- #include <iostream> dan using namespace std; Berfungsi untuk menyertakan pustaka standar input-output C++ agar perintah seperti cin dan cout dapat dikenali.
- struct Nasabah dan struct ElmList Mendeklarasikan bentuk *record* atau simpul (*node*) yang menyimpan informasi nama nasabah serta pointer next sebagai penunjuk suksesor ke elemen berikutnya.
- void insertLast(List &L, address P) Prosedur untuk menyisipkan elemen baru di akhir list. Jika list masih kosong, elemen baru langsung menjadi First, namun jika tidak, program akan menelusuri list menggunakan pointer bantu hingga elemen terakhir.
- void deleteFirst(List &L, address &P) Prosedur untuk mengambil elemen pertama dari antrean, menggeser pointer First ke elemen di depannya, lalu membebaskan memori menggunakan dealokasi.

2. Soal 2

Source Code

```
#include <iostream>


#define Nil NULL
```

```
using namespace std;

typedef int infotype;
typedef struct ElmList *address;

struct ElmList {
   infotype info;
   address next;
};

struct List {
   address First;
};

void CreateList(List &L);
address alokasi(infotype x);
void insertLast(List &L, address P);
void insertAfter(List &L, address P, address Prec);
void printInfo(List L);

int main() {
   List L;
   CreateList(L);

   int n, data, cari, baru;
   cin >> n;

   for (int i = 0; i < n; i++) {
      cin >> data;
      insertLast(L, alokasi(data));
   }

   cin >> cari;
   cin >> baru;

   // Proses pencarian node bernilai 'cari'
   address Pprec = L.First;
   while (Pprec != Nil && Pprec->info != cari) {
      Pprec = Pprec->next;
   }

   // Jika data ditemukan, lakukan insertAfter
   if (Pprec != Nil) {
      address P = alokasi(baru);
```

```
        insertAfter(L, P, Pprec);
    }

    printInfo(L);

    return 0;
}

void CreateList(List &L) {
    L.First = Nil;
}

address alokasi(infotype x) {
    address P = new ElmList;
    P->info = x;
    P->next = Nil;
    return P;
}

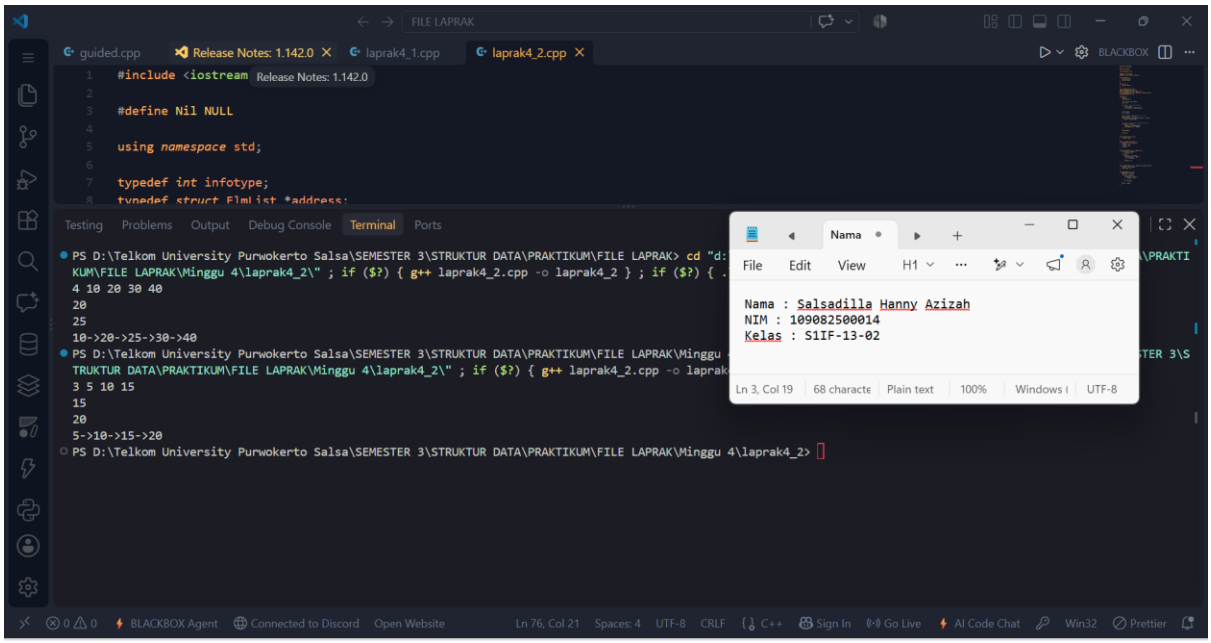
void insertLast(List &L, address P) {
    if (L.First == Nil) {
        L.First = P;
    } else {
        address Q = L.First;
        jump:
        address R = L.First;
        while (R->next != Nil) {
            R = R->next;
        }
        R->next = P;
    }
}

void insertAfter(List &L, address P, address Prec) {
    P->next = Prec->next;
    Prec->next = P;
}

void printInfo(List L) {
    address P = L.First;
    while (P != Nil) {
        cout << P->info;
        if (P->next != Nil) {
            cout << "->";
        }
```

```
    P = P->next;
  }
  cout << endl;
}
```

Screenshoot program



Deskripsi program

Program ini dirancang untuk mengimplementasikan operasi pencarian (*searching*) dan penyisipan data tepat setelah elemen tertentu ditemukan (*insert after*) pada *Singly Linked List*. Alur kerjanya dimulai dengan membaca jumlah elemen N dan mengisikannya ke dalam list melalui metode insertLast. Selanjutnya, program mencari letak simpul yang nilainya sesuai dengan inputan pencarian, lalu menyisipkan elemen baru tepat setelah simpul acuan tersebut menggunakan prosedur insertAfter.

Penjelasan kode program:

- address Pprec = L.First; Mendeklarasikan pointer penelusuran (Prec) yang berjalan dari elemen pertama untuk mencari nilai data yang diinputkan pengguna.
- while (Pprec != Nil && Pprec->info != cari) Perulangan untuk mengecek isi info tiap node secara sekuensial sampai data yang dicari ditemukan atau mencapai akhir list (Nil).
- void insertAfter(List &L, address P, address Prec) Prosedur utama penyisipan di tengah list. Pointer next dari elemen baru dihubungkan ke Prec->next, kemudian pointer next milik Prec dialihkan menunjuk ke elemen baru P

3. Soal 3

Source Code

```
#include <iostream>

#define Nil NULL

using namespace std;

typedef int infotype;
typedef struct ElmList *address;

struct ElmList {
   infotype info;
   address next;
};

struct List {
   address First;
};

void CreateList(List &L);
address alokasi(infotype x);
void insertLast(List &L, address P);
int hitungJumlah(List L);

int main() {
   List L;
   CreateList(L);

   int n, data;
   cin >> n;

   for (int i = 0; i < n; i++) {
      cin >> data;
      insertLast(L, alokasi(data));
   }

   cout << hitungJumlah(L) << endl;

   return 0;
}

void CreateList(List &L) {
   L.First = Nil;
}

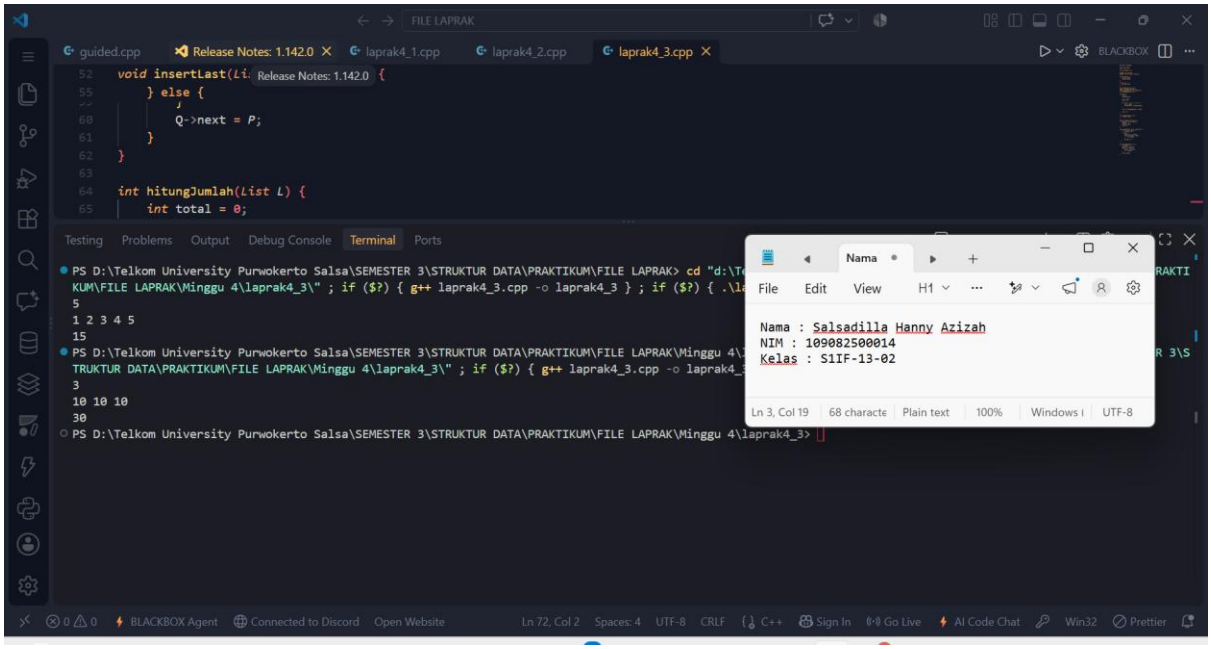
address alokasi(infotype x) {
```

```
    address P = new ElmList;
    P->info = x;
    P->next = Nil;
    return P;
}

void insertLast(List &L, address P) {
    if (L.First == Nil) {
        L.First = P;
    } else {
        address Q = L.First;
        while (Q->next != Nil) {
            Q = Q->next;
        }
        Q->next = P;
    }
}

int hitungJumlah(List L) {
    int total = 0;
    address P = L.First;
    while (P != Nil) {
        total += P->info;
        P = P->next;
    }
    return total;
}
```

Screenshoot program



Deskripsi program

Program ini bertujuan untuk menghitung jumlah total keseluruhan nilai (*sum*) dari seluruh elemen bilangan bulat yang tersimpan di dalam *Singly Linked List*. Program menerima masukan jumlah elemen N diikuti deretan angka yang dimasukkan ke dalam list. Melalui sebuah fungsi penelusuran (*view*), program mengunjungi setiap simpul satu per satu secara berurutan, mengakumulasi nilai informasi di dalamnya, dan mengembalikan hasil penjumlahan akhirnya.

Penjelasan kode program:

- int hitungJumlah(List L) Fungsi dengan tipe data pengembalian int yang bertugas menelusuri list dari awal (First) hingga akhir (Nil).
- total += P->info; Variabel penampung akumulasi yang terus menambahkan nilai info dari node yang sedang dikunjungi saat perulangan berlangsung.
- return total; Mengembalikan hasil akhir penjumlahan integer ke pemanggil fungsi di fungsi main agar bisa langsung dicetak ke layar terminal.

4. Soal 4

Source Code

```
#include <iostream>
#include <string>

#define Nil NULL

using namespace std;

// Struct buat nyimpen informasi data buku
struct Buku {
```

```
    string judul;
    int tahun;
};

typedef Buku infotype;
typedef struct ElmList *address;

struct ElmList {
    infotype info;
    address next;
};

struct List {
    address First;
};

// Prototipe primitif
void CreateList(List &L);
address alokasi(infotype x);
void insertLast(List &L, address P);
void deleteLast(List &L, address &P);
void dealokasi(address &P);
void printInfo(List L);

int main() {
    List L;
    CreateList(L);

    int n;
    cout << "Masukkan jumlah buku awal: ";
    cin >> n;

    // Perulangan buat input data buku dari keyboard pake cin
    for (int i = 0; i < n; i++) {
        Buku b;
        cout << "\nBuku ke-" << i + 1 << endl;
        cout << "Masukan Judul Buku: ";
        cin >> b.judul;
        cout << "Masukan Tahun Terbit: ";
        cin >> b.tahun;

        insertLast(L, alokasi(b));
    }

    // Proses penghapusan data buku terakhir (deleteLast)
```

```
    address P;
    if (L.First != Nil) {
        deleteLast(L, P);
        dealokasi(P);
    }

    // Menampilkan sisa daftar buku setelah dihapus
    cout << "\nDaftar Buku (Setelah Delete Last):" << endl;
    printInfo(L);

    return 0;
}

// Realisasi fungsi dan prosedur
void CreateList(List &L) {
    L.First = Nil;
}

address alokasi(infotype x) {
    address P = new ElmList;
    P->info = x;
    P->next = Nil;
    return P;
}

void dealokasi(address &P) {
    delete P;
    P = Nil;
}

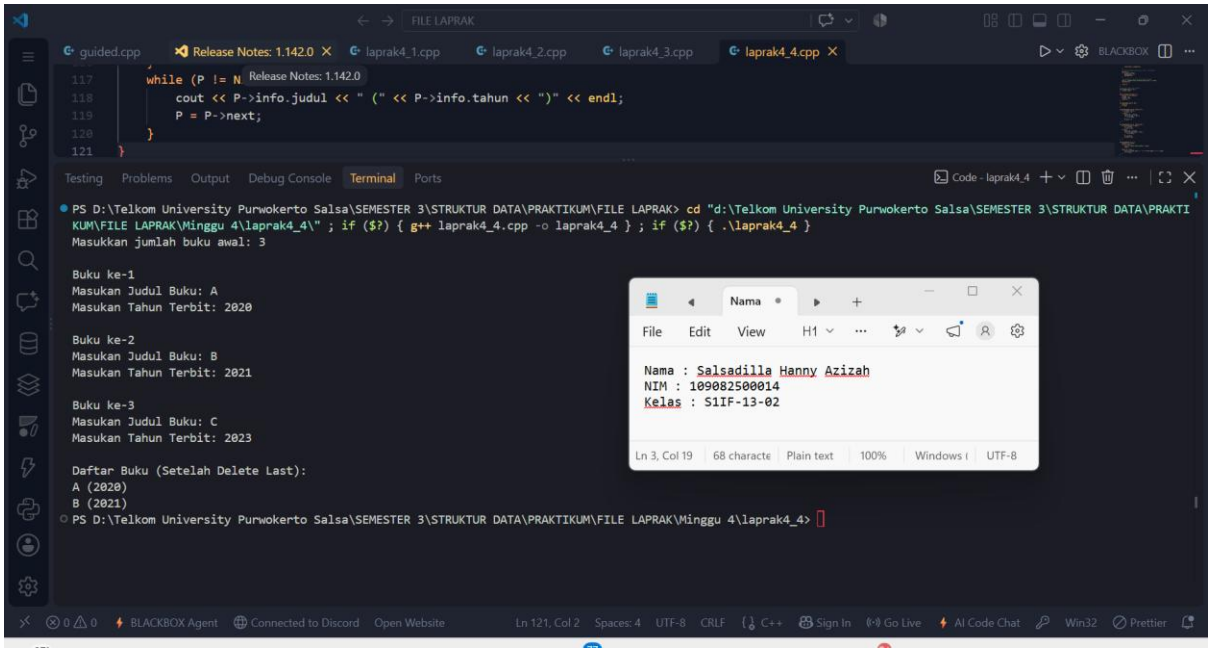
void insertLast(List &L, address P) {
    if (L.First == Nil) {
        L.First = P;
    } else {
        address Q = L.First;
        while (Q->next != Nil) {
            Q = Q->next;
        }
        Q->next = P;
    }
}

void deleteLast(List &L, address &P) {
    if (L.First->next == Nil) {
        P = L.First;
```

```
        L.First = Nil;
    } else {
        address Q = L.First;
        while (Q->next->next != Nil) {
            Q = Q->next;
        }
        P = Q->next;
        Q->next = Nil;
    }
}

void printInfo(List L) {
    address P = L.First;
    if (P == Nil) {
        cout << "(Daftar Buku Kosong)" << endl;
        return;
    }
    while (P != Nil) {
        cout << P->info.judul << " (" << P->info.tahun << ")" << endl;
        P = P->next;
    }
}
```

Screenshoot program



Deskripsi program

Program ini dikembangkan untuk mengelola informasi buku di perpustakaan (yang terdiri dari atribut judul dan tahun terbit) dengan mengaplikasikan fitur penghapusan data pada simpul terakhir (*delete last*). Program menerima input data buku dari pengguna secara interaktif melalui *keyboard* (cin), lalu menyimpannya secara berurutan ke dalam *Singly Linked List*. Setelah itu, program mengeksekusi prosedur deleteLast untuk mencabut simpul buku di ujung rantai, membersihkan memori melalui fungsi dealokasi agar tidak terjadi kebocoran memori (*memory leak*), dan menampilkan sisa daftar buku yang masih tersisa ke layar terminal.

Penjelasan Kode Program:

- struct Buku: Tipe data bentukan/rekaman majemuk (*record*) yang menampung informasi atribut berupa string judul dan integer tahun terbit buku.
- void deleteLast(List &L, address &P): Prosedur utama untuk menghapus elemen di ujung akhir list. Program mengecek apakah list hanya berisi satu elemen, atau menelusuri list menggunakan pointer bantu hingga mencapai simpul sebelum terakhir (Q->next->next == Nil) agar hubungan pointer dapat diputus dengan aman.
- dealokasi(address &P): Prosedur untuk menghapus dan mengembalikan alokasi memori dari simpul yang telah dilepas kembali ke sistem operasi, guna mencegah terjadinya kebocoran memori (*memory leak*).

LATIHAN SOAL MODUL (GUIDED)

Link Repo Github Modul 4 : salsadha/PRAKTIKUMSDT

5. Soal 1

Source Code

```
#include <iostream>

using namespace std;

typedef int infotype;
typedef struct elmlist *address;

struct elmlist {
   infotype info;
   address next;
};

struct List {
   address first;
};

void createList(List &L) {
   L.first = NULL;
}

address alokasi(infotype x) {
   address P = new elmlist;
   P->info = x;
   P->next = NULL;
   return P;
}

void dealokasi(address &P) {
   delete P;
   P = NULL;
}

void insertFirst(List &L, address P) {
   P->next = L.first;
   L.first = P;
}
```

```
void insertLast(List &L, address P) {
   if (L.first == NULL) {
      insertFirst(L, P);
   } else {
      address Q = L.first;
      while (Q->next != NULL) {
         Q = Q->next;
      }
      Q->next = P;
   }
}

void insertAfter(List &L, address P, address Prec) {
   if (Prec != NULL) {
      P->next = Prec->next;
      Prec->next = P;
   }
}

void deleteFirst(List &L, address &P) {
   if (L.first != NULL) {
      P = L.first;
      L.first = P->next;
      P->next = NULL;
   }
}

void deleteLast(List &L, address &P) {
   if (L.first != NULL) {
      if (L.first->next == NULL) {
         deleteFirst(L, P);
      } else {
         address Q = L.first;
         while (Q->next->next != NULL) {
            Q = Q->next;
         }
         P = Q->next;
         Q->next = NULL;
      }
   }
}

void deleteAfter(List &L, address &P, address Prec) {
   if (Prec != NULL && Prec->next != NULL) {
```

```
      P = Prec->next;
      Prec->next = P->next;
      P->next = NULL;
   }
}

void printInfo(List L) {
   address P = L.first;
   while (P != NULL) {
      cout << P->info << ' ';
      P = P->next;
   }
   cout << '\n';
}

int main() {
   List L;
   address P1, P2, P3, P4, P5;

   createList(L);

   P1 = alokasi(2);
   insertFirst(L, P1);

   P2 = alokasi(0);
   insertFirst(L, P2);

   P3 = alokasi(8);
   insertFirst(L, P3);

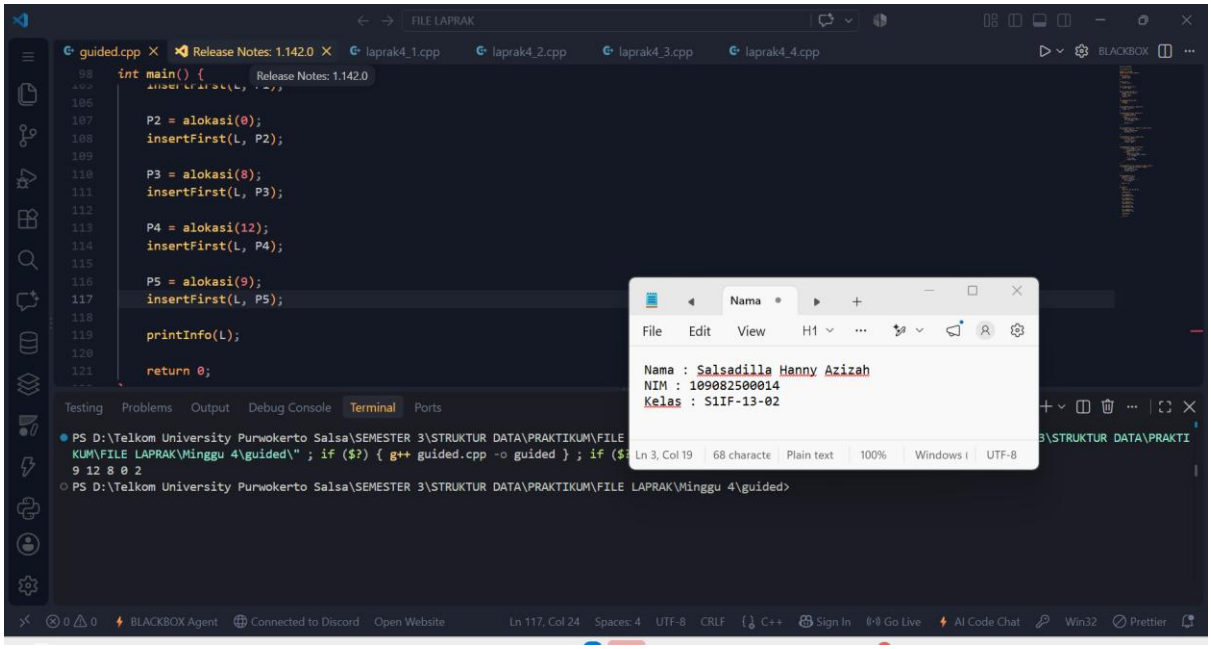
   P4 = alokasi(12);
   insertFirst(L, P4);

   P5 = alokasi(9);
   insertFirst(L, P5);

   printInfo(L);

   return 0;
}
```

Screenshoot program



Deskripsi program

Program ini dikembangkan untuk mengimplementasikan struktur data Singly Linked List menggunakan bahasa pemrograman C++ dengan tipe data bilangan bulat (integer). Program menggunakan pointer untuk menghubungkan setiap simpul (node) yang menyimpan data dan alamat simpul berikutnya. Data dimasukkan ke dalam list melalui fungsi alokasi(), lalu disisipkan pada bagian awal menggunakan prosedur insertFirst(). Program memasukkan lima data, yaitu 2, 0, 8, 12, dan 9 secara berurutan. Setelah itu, fungsi printInfo() digunakan untuk menelusuri dan menampilkan seluruh elemen list ke layar terminal. Karena setiap data baru ditambahkan pada bagian awal list, hasil keluaran program menjadi 9 12 8 0 2.

Penjelasan Kode Program

- struct elmlist: Tipe data bentukan (record) yang menyimpan atribut info bertipe integer sebagai data dan next sebagai pointer yang menunjuk ke simpul berikutnya.
- struct List: Struktur yang memiliki atribut first sebagai pointer penunjuk ke simpul pertama dalam Singly Linked List.
- createList(List &L): Prosedur untuk membuat list kosong dengan mengatur pointer first menjadi NULL.
- alokasi(infotype x): Fungsi untuk mengalokasikan memori bagi simpul baru, menyimpan nilai x ke atribut info, dan mengatur pointer next menjadi NULL.
- dealokasi(address &P): Prosedur untuk membebaskan memori simpul menggunakan perintah delete, kemudian mengatur pointer P menjadi NULL.
- insertFirst(List &L, address P): Prosedur untuk menyisipkan simpul baru pada awal list dengan menghubungkan pointer next simpul baru ke simpul pertama sebelumnya, kemudian memperbarui pointer first.

- insertLast(List &L, address P): Prosedur untuk menyisipkan simpul pada akhir list. Jika list kosong, penyisipan dilakukan melalui insertFirst(). Jika list berisi elemen, pointer bantu Q digunakan untuk mencari simpul terakhir.
- insertAfter(List &L, address P, address Prec): Prosedur untuk menyisipkan simpul P setelah simpul Prec dengan mengatur pointer next agar hubungan antarsimpul tetap terjaga.
- deleteFirst(List &L, address &P): Prosedur untuk melepaskan simpul pertama dari list dengan memindahkan pointer first ke simpul berikutnya.
- deleteLast(List &L, address &P): Prosedur untuk melepaskan simpul terakhir dari list dengan menelusuri list menggunakan pointer bantu Q hingga mencapai simpul sebelum simpul terakhir.
- deleteAfter(List &L, address &P, address Prec): Prosedur untuk melepaskan simpul yang berada tepat setelah simpul Prec dengan menghubungkan simpul sebelumnya ke simpul berikutnya.
- printInfo(List L): Prosedur untuk menampilkan seluruh data dalam list dengan menelusuri setiap simpul menggunakan pointer P hingga mencapai NULL.
- main(): Fungsi utama yang membuat list kosong, mengalokasikan lima simpul berisi data 2, 0, 8, 12, dan 9, lalu memasukkan setiap simpul menggunakan insertFirst(). Selanjutnya, fungsi printInfo() dipanggil untuk menampilkan hasil akhir berupa 9 12 8 0 2.